4th Credit Hour Assignment: LLM Agents

Version: 1.0
Updated: Sep 24

1. Objective:

LLM coding agents (Devin, OpenHands, Moatless, SWE-Agent, Claude Code, Gemini CLI, Codex, etc.) have been increasingly adopted in practice. This 4th-credit-hour assignment aims to help you understand their internal design and learn how to use them more smoothly in your daily development. More specifically, you will work in a team to study an existing minimal LLM coding agent framework, add more complex features to improve performance, evaluate the system on a subset of SWE-Bench problems, and write a technical report.

2. Important Deadlines:

- **GitHub Repository Setup: Oct 5**
- **Midterm Milestone: Nov 9**
- **Final: Dec 10**

3. Introduction:

Large Language Model (LLM) coding agents are AI systems that can autonomously read, write, and modify code to resolve real-world software engineering tasks. Unlike simple code completion tools, they combine reasoning with the ability to use external resources and tools, allowing them to act more like software engineers than passive assistants.

A common way to evaluate these agents is through SWE-bench, a benchmark built from real GitHub issues in open-source projects. Building on this, several systems have been proposed — Agentless, SWE-agent, OpenHands, etc. — that demonstrate significant progress in enabling LLMs to tackle software engineering problems through better pipeline design, structured tool use, and multi-step reasoning. These advances highlight both the potential and the challenges of building truly agentic coding systems.

In this assignment, you will work with a simplified LLM agent system called mini-SWE-agent, which only has access to the bash tool. The goal is for you to gain hands-on experience with how such systems are designed, understand their strengths and limitations, and propose improvements to extend the capabilities of the mini-SWE-agent. For evaluation, we will focus on a subset of SWE-bench: Verified-50.

You should carefully read the **entire** write-up to correctly set up the project, implement the required features, and finally write a technical report. This assignment is very open-ended, and we truly believe outstanding projects can lead to research publications and real-world impact.

4. Preparation:

4.1 Team

You will work with the same team as for the Android App Term project. We do not enforce roles or require weekly meetings for the 4th credit hour assignment; however, everyone is expected to contribute equally to the assignment and make consistent progress throughout the semester. At the end of the semester, we will release a dedicated peer evaluation form for this assignment and will adjust individual scores based on the results.

4.2 Python

For this assignment, you will work with Python and need to have Python version 3.12 or higher installed on your system. We recommend using Miniconda to install an isolated Python environment for this assignment:

- Miniconda installation: https://www.anaconda.com/docs/getting-started/miniconda/install
- Python environment: https://www.anaconda.com/docs/getting-started/working-with-conda/environments

Example commands to set up a new Miniconda environment for the assignment:

```shell
$ conda create -n sweagent python=3.12 -y
$ conda activate sweagent
```

You don't need to be an expert in Python to succeed in this assignment, but you should know the basics. Feel free to consult AI (ChatGPT, Claude, Gemini…) for basic Python questions. Here are also some detailed tutorials:

- https://docs.python.org/3/tutorial/index.html
- https://www.geeksforgeeks.org/python/python-programming-language-tutorial/
- https://www.pythoncheatsheet.org/

4.3 API Key

You can use Gemini LLM as the model for your agent. Gemini LLM offers a **free tier** with usage limits, enabling experimentation with the model at no cost. To get an API key for Gemini, visit https://aistudio.google.com/apikey and click on "Create API Key" in the top-right corner.

- For more documentation: https://ai.google.dev/gemini-api/docs
- For pricing information: https://ai.google.dev/pricing
- For rate limits: https://ai.google.dev/gemini-api/docs/rate-limits#free-tier

Important note: we are **NOT** asking you to pay for Gemini or any service for this assignment or any part of the CS427 course. ***Make sure you do not associate billing information with the API key to keep the key in the free tier.*** If you have any trouble with free services, please let us know.

Once you have obtained the API key, you can verify that it works by running:

```shell
$ export GEMINI_API_KEY='your-api-key-here'

$ curl
"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:gener
ateContent" \
  -H 'Content-Type: application/json' \
  -H "X-goog-api-key: $GEMINI_API_KEY" \
  -X POST \
  -d '{
    "contents": [
      {
        "parts": [
          {
            "text": "Explain what are crucial compoenents for LLM agents in a
few words."
          }
        ]
      }
    ]
  }'
```

Important note: **NEVER** share the API key publicly or hardcode it in code.

4.4 GitHub Repository

We have prepared a copy of `mini-swe-agent` with some additional features implemented: https://github.com/CS427-FALL-2025/mini-swe-agent. You should download and unzip the codebase and use it to set up your team's own GitHub repository. Your team needs to set up a private GitHub repository named `cs427-agent-teamXY`, where `XY` is your team number (e.g., `cs427-agent-team01` for Team 1).

You should grant collaborator access to (1) all team members and (2) your dedicated TA. Failing to give access to the course staff results in losing **100% points.**

4.5 mini-SWE-agent

Once your team's `mini-swe-agent` codebase is ready, you can run the following commands to download dependencies and install `mini-swe-agent` on your system:

```shell
$ cd mini-swe-agent
$ conda activate sweagent # Adjust the env name based on your setup
$ pip install -e .
```

Next, run `mini-swe-agent`, choose the model `gemini/gemini-2.5-flash`, and provide your API key:

```shell
$ mini
# Enter your default model (example: claude-...): gemini/gemini-2.5-flash
# Enter your API key name (e.g., ANTHROPIC_API_KEY): GEMINI_API_KEY
# Enter your API key value (e.g., sk-1234567890): your-api-key
```

If you need to update the model and key, run `mini-extra config setup`
Here is an example to run `mini-swe-agent`:

```shell
$ mini -m gemini/gemini-2.5-flash -t "Show my the content of README.md"
```

Note: Be cautious when using the `yolo` mode, as it allows the model to run arbitrary commands on your machine :)

4.6 SWE-bench

Now we walk through the process of using `mini-swe-agent` to generate patches for `SWE-bench` problems and evaluate the patches.
For this assignment, we will evaluate your system on a small subset of `SWE-bench Verified`, named `Verified-50`, introduced by this work. However, you are free to work on `SWE-bench Verified` or the full `SWE-bench` dataset to help improve your system. The `Verified-50` and several variants have been included in the provided codebase, and the Task section will introduce more details.

4.6.1 Generate Patches

You will need to use `mini-extra` to generate patches. You can run `mini-extra swebench --help` for all available arguments. Here is an example command to work on the first instance

in the `Verified-50` dataset:

```
Shell
$ mini-extra swebench \
    --model gemini/gemini-2.5-flash \
    --subset verified-50 \
    --slice 0:1 \ # Just run on first instance
    --output example_gen # Adjust the output directory accordingly
```

Note: the free tier of `gemini-2.5-flash` has a low rate limit (10 requests per minute), thus you should not use the `--worker` flag.

4.6.2 Evaluate Patches

After successfully generating the patches, you will see a `preds.json` file in the output directory, which contains all the required information for running `SWE-bench`.

Before the first time you run the evaluation, you should run the following command to install `SWE-bench` on your system:

```
Shell
$ cd mini-swe-agent/SWE-bench
$ conda activate sweagent # Adjust the env name based on your setup
$ pip install -e .
```

Then, run this command to evaluate the `preds.json` file:

```
Shell
$ python -m swebench.harness.run_evaluation \
    --dataset_name princeton-nlp/SWE-Bench_Verified \
    --predictions_path PATH_TO_PREDS_JSON \
    --run_id example_eval
```

This will generate a file `gemini__gemini-2.5-flash.example_eval.json` (the name might change if you used different models during generation). In the file, you can ignore the `total_instances: 500` field since we are only working on a subset of the instances. For the final report, you should report the `resolved_instances` and `unresolved_instances` numbers.

5. Tasks:

5.1 Tools

`mini-swe-agent` only supports the bash tool. Your task is to enhance the capabilities of `mini-swe-agent` by developing and integrating new, lightweight tools to expand the range of operations the agent can perform beyond executing bash commands, while maintaining the agent's compact design and simple control flow. You can refer to the `cs427_tools_extension.md` file in the template repository for how to add basic tools into the `mini-swe-agent` framework. We have provided a basic file reader tool as a reference for you to get started.

5.1.1 Requirements:

- **Implement the write tool**: follow the read tool and implement a basic file writer
- **Refine the read tool**: change the read tool to support only reading certain lines
    - e.g., the model could provide line numbers, line number ranges, first N lines, last N lines…)
    - You are free to decide how you want to refine the tool, but you need to explain your decision in the report
    - [Optional] You can also improve the writer tool similarly.
- **At least four new, distinct tools**: excluding the above read and write, you need to implement four other tools that can help model to solve more `SWE-bench` problems. You have full freedom to implement any tool as long as it can improve performance.

5.1.2 Possible Tool Ideas:

This task is fully open-ended, and you should be creative. Here are just a few examples and resources to get you started:

- **`search_file_content`**: search patterns in a file and show its surrounding context
- **`replace`**: search and replace certain words/sentences in a file (or apply a diff patch)
- **`read_many_files`**: read several files at the same time and provide a succinct representation
- **`todo_write`**: maintain and update a TODO list
- **...**

Tools in more advanced coding agents:

- Claude Code: https://docs.claude.com/en/docs/claude-code/settings#tools-available-to-claude
- Gemini CLI: https://google-gemini.github.io/gemini-cli/docs/tools/
- Cline: https://docs.cline.bot/exploring-clines-tools/cline-tools-guide#available-tools

5.2 [Optional] Memory

Note: memory mechanism is ***fully optional and open-ended***. Most problems in our evaluation dataset don't need a memory mechanism to be solved, but memory could help with the most challenging instances, and is also a good concept to explore for curious teams :)

While tools can enable LLM coding agents to perform well on many problems, agents can still struggle when the context grows too large or contains too much unrelated information, or just when the codebase is too large. Several kinds of memory mechanisms could help agents perform better on long and complex tasks by smartly selecting what information to put in the context. Here are two examples in Claude Code:

- **Memory Compression**: When the agents have gone through many iterations of thinking and tool calls, the large context could be replaced by a smaller summarization.
    - This type of memory mechanism requires dynamically updating the message history to make it shorter and more relevant.
- **Hierarchical Memory**: For a large directory or a large file, one could save a succinct summary of the content so that the agent can refer to it later without spending tokens to read the original content.
    - This type of memory mechanism provides shorter and higher-quality information in each step's prompt, based on what content the agent tries to access.

We have provided a simple message history compression example in the template repo. You can check the `cs427_memory.md` in the repo for more details. This serves as an example showing how the message history could be modified, but you have full freedom to implement other kinds of memory mechanisms.

Some more possible ideas:

- **`memory_search`**: search through conversation history using keywords, patterns, or semantic similarity to find relevant past information
- **`tool_feeback_compression`**: intelligently compress feedback from tool calls to preserve only the useful information (e.g., most of the content in a long error message could be irrelevant)

You can check out advanced coding agents for more ideas. Here are some references to get you started:

- https://claudelog.com/faqs/what-is-claude-code-auto-compact/
- https://medium.com/@tl_99311/claude-codes-memory-working-with-ai-in-large-codebases-a948f66c2d7e
- https://github.com/SWE-agent/SWE-agent/blob/main/sweagent/agent/agents.py
- https://github.com/SWE-agent/SWE-agent/blob/main/sweagent/agent/models.py
- https://github.com/SWE-agent/SWE-agent/blob/main/sweagent/agent/history_processors.py

5.3 Evaluation

In the template repo, we have provided access to running the `mini-SWE-agent` on the `Verified-50` and its subsets. You can select the subset using the `--subset` flag while running `mini-extra swebench … --subset verified-XX …`
The variants are:

- `verified-40`: easiest 40 instances
- `verified-45-extra`: next 5 harder instances
- `verified-50-extra`: top 5 most challenging instances that may need many tool calls
- `verified-45`: `verified-40 + verified-45-extra`
- `Verified-50`: `verified-40 + verified-45-extra + verified-50-extra`

You are **<u>required</u>** to evaluate your improved system on the `verified-40` subset and achieve a good performance to get a full score for this assignment. However, we encourage all teams to work on the more challenging instances and offer **<u>bonus points</u>** for resolving them.

The performance is worth 8 points for this assignment (and your overall score in the class). The detailed evaluation criteria are:

- `Verified-40`
  - < 18 ⇒ 0 points
  - 18 ⇒ 4 points
  - 19 ⇒ 5 points
  - 20 ⇒ 6 points
  - 21 ⇒ 7 points
  - 22 ⇒ 8 points
  - > 22 ⇒ 0.25 points for each additional one
- `verified-45-extra`
  - 1/5 ⇒ No extra points
  - Resolving each additional one earns 0.5 extra points
- `verified-50-extra`
  - 1/5 ⇒ No extra points
  - Resolving each additional one earns 1 extra point

You are required to save and submit all the generated files from the `mini-SWE-agent` and the evaluation file generated by `SWE-bench`. See the Deliverables section for the required location and format.

<u>Warning:</u> You **MUST** use the `gemini-2.5-flash` model for evaluation. You **MUST NOT** change the instance IDs in the dataset or modify any part of `SWE-bench`. You **MUST NOT** hardcode any information about solutions in code or prompts. Any such violation will result in 0 points for the entire assignment.

Note: the extra credit can make you earn more than 100% for this assignment, making up for any points

5.4 Project Report

Regardless of the performance, each team should also write a **midterm milestone report** and a **final report** to discuss their design choices and learnings. Both reports should be PDF files uploaded to your GitHub repository. Check the Deliverable section for the required location.

We do not enforce a specific format for the report, but both the midterm and final reports must contain the following sections and content:

- **Performance:** At the beginning of your report, include a table for the performance of your improved system. Any mismatch between your report and the saved generation & evaluation results will result in 0 points for the whole assignment.
    - Copy the following table or create a similar one:

Subset	Resolved Count	Resolved ID
verified-40	XX	django__django-11532…
verified-45-extra	Optional	Optional
verified-50-extra	Optional	Optional

- **Design**: for each implemented tool and memory operation (including the improved read & write tools):
    - Why do you think it is useful? Are there alternatives, and why are they not chosen?
    - How did you implement it? Describe any challenges you faced or any sacrifices you made.
    - Was it actually useful? Provide **at least one case study** demonstrating how the operation/tool helped or hindered generation.
- **[Optional] Learnings**: if your team tries to tackle the more challenging 10 instances but fails to resolve any, you are also encouraged to document your effort for **potential** bonus points. Describe what you tried, why it did not help, and how you would further improve the system if you had more time and resources.

For the midterm report, you should follow the format described above, but we only ask for:

- **Performance:** You should include the table in the report with your system's performance numbers and the required files in your repo. However, it will NOT be graded. The performance score will solely depend on your final report.
- **Design:** You are only required to write the designs for the write tool.
- **Learnings:** still optional

6. Deliverables:

Please follow the tasks explained under the **Tasks** section to implement the requested features and write the final report. Your deliverables are:

- **Codebase & Report**
  - The codebase should be accessible to course staff through GitHub by the deadline of the GitHub setup
  - **For the midterm,** the codebase must contain a `deliverables_midterm` directory by the deadline of the midterm milestone.
  - **For the final**, the codebase must contain a `deliverables_final` directory by the deadline of the final submission.

Shell

deliverables_midterm

```
├── gemini__gemini-2.5-flash.eval.json     # Evaluation Result
├── generation
│   ├── django__django-10880              # Generation traces for instances
│   ├── django__django-11095
│   ├── django__django-11265
│   │    ...
│   └── preds.json                        # Generated patches
└── report.pdf                             # Midterm Report
```

deliverables_final

```
├── gemini__gemini-2.5-flash.eval.json     # Evaluation Result
├── generation
│   ├── django__django-10880              # Generation traces for instances
│   ├── django__django-11095
│   ├── django__django-11265
│   │    ...
│   └── preds.json                        # Generated patches
└── report.pdf                             # Final Report
```

- **Peer Evaluation Form:**
  - At the end of the semester, we will release a peer evaluation form for this assignment. Scores for individual students might be adjusted based on the peer evaluation results.

7. Grading rubric:

Activity	Points
Setup	1
Midterm Codebase	3
Final Codebase	5
Midterm Report	3
Final Report	5
Final Performance	8
Total	25